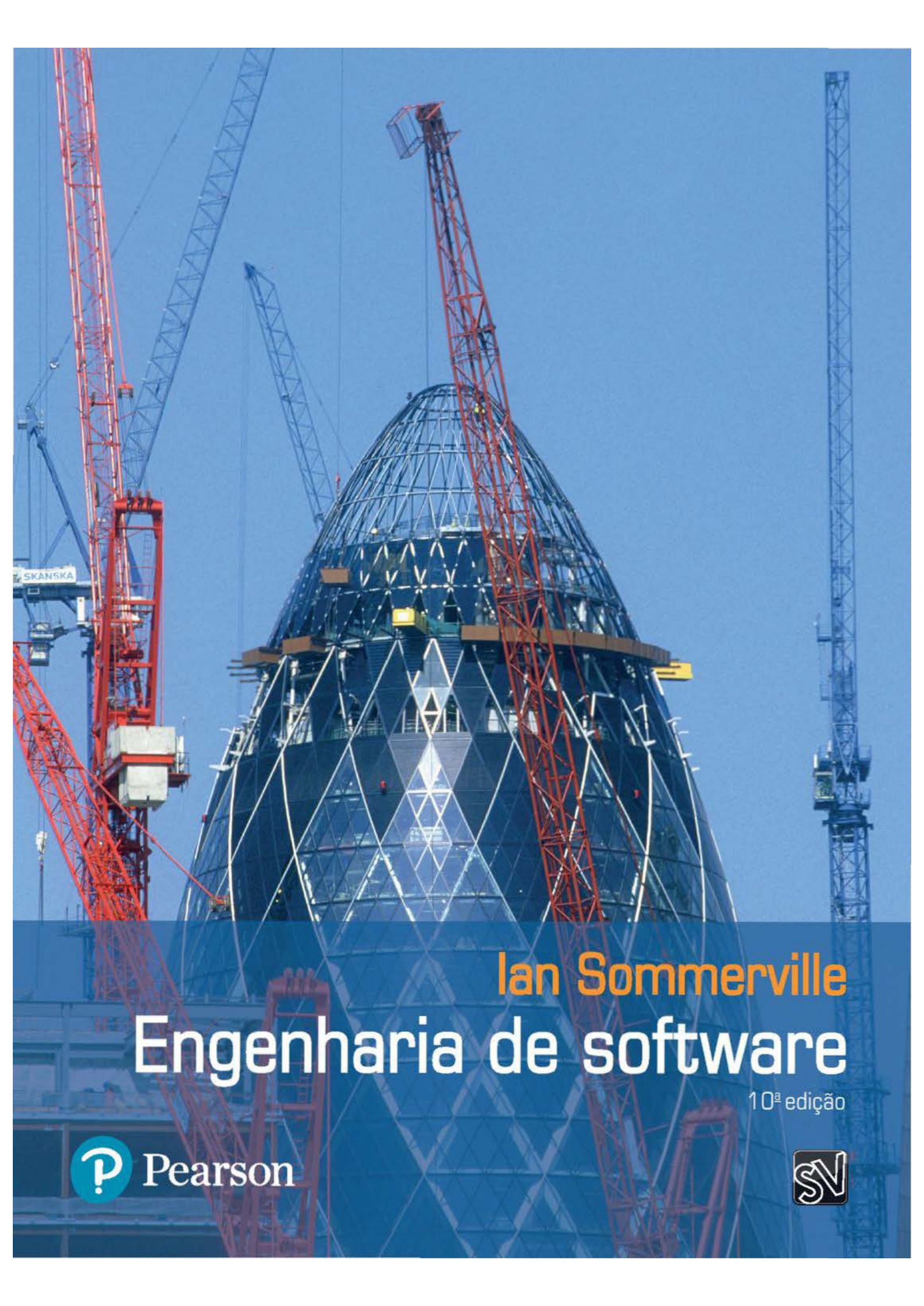




Ian Sommerville

Engenharia de software

10ª edição

 Pearson



Engenharia de software

10ª edição

Ian Sommerville

Engenharia de software

10ª edição

Tradução

Luiz Claudio Queiroz

Revisão técnica

Prof. Dr. Fábio Levy Siqueira
Escola Politécnica da Universidade de
São Paulo (Poli-USP)
Departamento de Engenharia de
Computação e Sistemas Digitais (PCS)



© 2016, 2011, 2006 by Pearson Higher Education, Inc.

© 2019 by Pearson Education do Brasil Ltda.

Todos os direitos reservados. Nenhuma parte desta publicação poderá ser reproduzida ou transmitida de qualquer modo ou por qualquer outro meio, eletrônico ou mecânico, incluindo fotocópia, gravação ou qualquer outro tipo de sistema de armazenamento e transmissão de informação, sem prévia autorização por escrito da Pearson Education do Brasil.

VICE-PRESIDENTE DE EDUCAÇÃO	Juliano Costa
CERENTE DE PRODUTOS	Alexandre Mattioli
SUPERVISORA DE PRODUÇÃO EDITORIAL	Silvana Afonso
COORDENADOR DE PRODUÇÃO EDITORIAL	Jean Xavier
EDIÇÃO	Mariana Rodrigues
ESTAGIÁRIO	Rodrigo Orsi
PREPARAÇÃO	Marcel Gugoni
REVISÃO	Lyvia Felix e Bruna Cordeiro
CAPA	Natália Gaio sobre o projeto original de Black Horse Designs (imagem de capa: Construction of the Gherkin, London, UK/Corbis)
DIAGRAMAÇÃO E PROJETO GRÁFICO	Casa de Ideias

Dados Internacionais de Catalogação na Publicação (CIP)
(Câmara Brasileira do Livro, SP, Brasil)

Sommerville, Ian
Engenharia de software / Ian Sommerville ; tradução Luiz Cláudio
Queiroz ; revisão técnica Fábio Levy Siqueira. -- 10. ed. -- São Paulo :
Pearson Education do Brasil, 2018.

Título original: Software engineering
ISBN 978-85-430-2497-4

1. Engenharia de software I. Siqueira, Fábio Levy. II. Título.

18-21869

CDD-005.1

Índice para catálogo sistemático:

1. Engenharia de software 005.1

Cibele Maria Dias - Bibliotecária - CRB-8/9427

2018

Direitos exclusivos cedidos à
Pearson Education do Brasil Ltda.,
uma empresa do grupo Pearson Education
Avenida Santa Marina, 1193
CEP 05036-001 - São Paulo - SP - Brasil
Fone: 11 2178-8609 e 11 2178-8653
pearsonuniversidades@pearson.com

Sumário

Prefácio	VII	Capítulo 7	Projeto e implementação	175
Parte 1		7.1	Projeto orientado a objetos usando UML	176
Capítulo 1	Introdução	7.2	Padrões de projeto	187
1.1	Desenvolvimento de software profissional	7.3	Questões de implementação	191
1.2	Ética da engenharia de software	7.4	Desenvolvimento de código aberto (<i>open source</i>)	197
1.3	Estudos de caso	Capítulo 8	Teste de software	203
Capítulo 2	Processos de software	8.1	Teste de desenvolvimento	208
2.1	Modelos de processo de software	8.2	Desenvolvimento dirigido por testes	218
2.2	Atividades do processo	8.3	Teste de lançamento	221
2.3	Lidando com mudanças	8.4	Teste de usuário	224
2.4	Melhoria de processo	Capítulo 9	Evolução de software	231
Capítulo 3	Desenvolvimento ágil de software	9.1	Processos de evolução	234
3.1	Métodos ágeis	9.2	Sistemas legados	237
3.2	Técnicas de desenvolvimento ágil	9.3	Manutenção de software	246
3.3	Gerenciamento ágil de projetos	Parte 2	Dependabilidade e segurança	257
3.4	Escalabilidade dos métodos ágeis	Capítulo 10	Dependabilidade de sistemas	259
Capítulo 4	Engenharia de requisitos	10.1	Propriedades de dependabilidade	261
4.1	Requisitos funcionais e não funcionais	10.2	Sistemas sociotécnicos	265
4.2	Processos de engenharia de requisitos	10.3	Redundância e diversidade	268
4.3	Elicitação de requisitos	10.4	Processos confiáveis	270
4.4	Especificação de requisitos	10.5	Métodos formais e dependabilidade	273
4.5	Validação de requisitos	Capítulo 11	Engenharia de confiabilidade	279
4.6	Mudança de requisitos	11.1	Disponibilidade e confiabilidade	282
Capítulo 5	Modelagem de sistemas	11.2	Requisitos de confiabilidade	285
5.1	Modelos de contexto	11.3	Arquiteturas tolerantes a defeitos	290
5.2	Modelos de interação	11.4	Programação para confiabilidade	297
5.3	Modelos estruturais	11.5	Medição da confiabilidade	304
5.4	Modelos comportamentais	Capítulo 12	Engenharia de segurança (<i>safety</i>)	311
5.5	Engenharia dirigida por modelos	12.1	Sistemas críticos em segurança	313
Capítulo 6	Projeto de arquitetura	12.2	Requisitos de segurança	316
6.1	Decisões de projeto de arquitetura	12.3	Processos de engenharia de segurança	323
6.2	Visões de arquitetura	12.4	Casos de segurança	332
6.3	Padrões de arquitetura			
6.4	Arquiteturas de aplicações			

Capítulo 13	Engenharia de segurança da informação (<i>security</i>)	343	Capítulo 20	Sistemas de sistemas	549
	13.1 Segurança da informação e dependabilidade	346		20.1 Complexidade de sistemas	552
	13.2 Segurança da informação e organizações	349		20.2 Classificação de sistemas de sistemas	556
	13.3 Requisitos de segurança da informação	352		20.3 Reducionismo e sistemas complexos	559
	13.4 Projeto de sistemas seguros	357		20.4 Engenharia de sistemas de sistemas	562
	13.5 Teste e garantia de segurança da informação	372		20.5 Arquitetura de sistemas de sistemas	567
Capítulo 14	Engenharia de resiliência	379	Capítulo 21	Engenharia de software de tempo real	577
	14.1 Cibersegurança	382		21.1 Projeto de sistemas embarcados	579
	14.2 Resiliência sociotécnica	387		21.2 Padrões de arquitetura para software de tempo real	587
	14.3 Projeto de sistemas resilientes	395		21.3 Análise temporal	593
Parte 3	Engenharia de software avançada	407		21.4 Sistemas operacionais de tempo real	598
Capítulo 15	Reúso de software	409	Parte 4	Gerenciamento de software	605
	15.1 O panorama do reúso	412	Capítulo 22	Gerenciamento de projetos	607
	15.2 <i>Frameworks</i> de aplicação	414		22.1 Gerenciamento de riscos	610
	15.3 Linhas de produtos de software	417		22.2 Gerenciamento de pessoas	616
	15.4 Reúso de sistemas de aplicação	424		22.3 Trabalho em equipe	621
Capítulo 16	Engenharia de software baseada em componentes	435	Capítulo 23	Planejamento de projetos	631
	16.1 Componentes e modelos de componentes	437		23.1 Precificação de software	634
	16.2 Processos de engenharia de software baseada em componentes	444		23.2 Desenvolvimento dirigido por plano	635
	16.3 Composição de componentes	451		23.3 Definição do cronograma de projeto	638
Capítulo 17	Engenharia de software distribuído	461		23.4 Planejamento ágil	644
	17.1 Sistemas distribuídos	463		23.5 Técnicas de estimativa	646
	17.2 Computação cliente-servidor	470		23.6 Modelagem de custos COCOMO	650
	17.3 Padrões de arquitetura de sistemas distribuídos	472	Capítulo 24	Gerenciamento da qualidade	663
	17.4 Software como serviço	484		24.1 Qualidade de software	666
Capítulo 18	Engenharia de software orientada a serviços	491		24.2 Padrões de software	669
	18.1 Arquitetura orientada a serviços	495		24.3 Revisões e inspeções	673
	18.2 Serviços RESTful	500		24.4 Gerenciamento da qualidade e desenvolvimento ágil	678
	18.3 Engenharia de serviços	503		24.5 Medição de software	680
	18.4 Composição de serviços	512	Capítulo 25	Gerenciamento de configuração	693
Capítulo 19	Engenharia de sistemas	521		25.1 Gerenciamento de versões	697
	19.1 Sistemas sociotécnicos	525		25.2 Construção de sistemas	704
	19.2 Projeto conceitual	533		25.3 Gerenciamento de mudanças	709
	19.3 Aquisição de sistemas	536		25.4 Gerenciamento de lançamentos	714
	19.4 Desenvolvimento de sistemas	539	Glossário		721
	19.5 Operação e evolução de sistemas	543	Índice de assuntos		733
			Índice de autores		753

Prefácio

O progresso na engenharia de software nos últimos 50 anos é surpreendente. Nossas sociedades não funcionariam sem grandes sistemas profissionais de software. Serviços de utilidade pública e infraestruturas nacionais — energia, comunicações e transportes —, todos dependem de sistemas informatizados complexos e de maior confiabilidade. O software permitiu-nos explorar o espaço e criar a *World Wide Web* — o sistema de informação mais significativo na história da humanidade. Smartphones e tablets são onipresentes, e toda uma indústria de desenvolvimento de aplicativos para esses dispositivos surgiu nos últimos anos.

Hoje, a humanidade enfrenta um conjunto exigente de desafios — as mudanças climáticas e o clima extremo, o declínio dos recursos naturais, a crescente população mundial a ser alimentada e abrigada, o terrorismo internacional e a necessidade de ajudar as pessoas idosas a levarem uma vida gratificante. Precisamos de novas tecnologias para nos ajudar a resolver esses desafios e, com certeza, o software terá um papel central nessas tecnologias. A engenharia de software é, portanto, de importância crítica para o nosso futuro neste planeta. Temos de continuar a educar os engenheiros de software e a desenvolver a disciplina para que atendamos à demanda por mais software e criemos os futuros sistemas cada vez mais complexos de que precisamos.

Naturalmente, ainda há problemas com os projetos de software. Os sistemas às vezes ainda são entregues com atraso e custam mais do que o esperado. Estamos criando sistemas de software cada vez mais complexos e não devemos nos surpreender por encontrarmos dificuldades ao longo do caminho. No entanto, não devemos deixar esses problemas esconderem os reais sucessos na engenharia de software e os impressionantes métodos de engenharia de software e tecnologias que foram desenvolvidos.

Este livro, em edições diferentes, já existe há mais de 30 anos, e esta edição é baseada nos princípios essenciais que foram estabelecidos na primeira edição:

1. Escrevo sobre a engenharia de software como ela é praticada na indústria, sem adotar uma postura doutrinadora quanto a abordagens específicas, como desenvolvimento ágil ou métodos formais. Na realidade, a indústria mistura técnicas como o desenvolvimento ágil e o desenvolvimento baseado em planos, e isso se reflete no livro.
2. Escrevo sobre o que conheço e entendo. Tive muitas sugestões para tópicos adicionais que poderiam ser abordados em mais detalhe, como o desenvolvimento de código aberto (*open source*), o uso da UML e a engenharia de software móvel. No entanto, realmente não conheço o suficiente sobre essas áreas. Meu próprio trabalho tem sido em confiabilidade e engenharia de sistemas, e isso se reflete na minha seleção de tópicos avançados para o livro.

Acredito que as principais questões para a engenharia de software moderna são gerenciar a complexidade, integrar a agilidade com outros métodos e garantir que

nossos sistemas sejam seguros (do ponto de vista de segurança da informação — *security*) e resilientes. Essas questões foram o vetor para as mudanças e acréscimos nesta nova edição do meu livro.

MUDANÇAS NA 10ª EDIÇÃO

Em resumo, as principais atualizações e acréscimos nesta 10ª edição do livro são:

- ▶ Atualizei extensivamente o capítulo sobre engenharia de software ágil, com novo material sobre Scrum. Atualizei outros capítulos, conforme o necessário, para refletir o uso crescente de métodos ágeis de engenharia de software.
- ▶ Adicionei novos capítulos sobre engenharia de resiliência, engenharia de sistemas e sistemas de sistemas.
- ▶ Reorganizei completamente três capítulos que abordam confiabilidade, segurança (*safety*) e segurança da informação (*security*).
- ▶ Adicionei novo material sobre serviços RESTful para o capítulo que aborda a engenharia de software orientada a serviços.
- ▶ Revisei e atualizei o capítulo sobre gerenciamento de configuração com novo material sobre sistemas de controle de versão distribuídos.
- ▶ Movi os capítulos sobre engenharia de software orientada a aspectos e melhoria de processos da versão impressa para o site do livro (em inglês).
- ▶ Novo material suplementar foi adicionado ao site, incluindo um conjunto de vídeos de apoio. Expliquei temas-chave em vídeo e recomendei vídeos do YouTube relacionados.

A estrutura em quatro partes do livro, introduzida nas edições anteriores, foi mantida, mas fiz alterações importantes em cada parte:

1. Na Parte 1, Introdução à engenharia de software, reescrevi completamente o Capítulo 3 (Desenvolvimento ágil de software), atualizando o conteúdo para refletir o crescente uso do Scrum. Um novo estudo de caso sobre um ambiente digital de aprendizagem foi adicionado ao Capítulo 1 e ele é usado em vários capítulos. Os sistemas legados são tratados com mais detalhes no Capítulo 9. Pequenas alterações e atualizações foram feitas em todos os outros capítulos.
2. A Parte 2, que aborda sistemas confiáveis, foi revisada e reestruturada. Em vez de uma abordagem orientada a atividades em que as informações sobre segurança (*safety*), segurança da informação (*security*) e confiabilidade estão espalhadas por vários capítulos, reorganizei o texto para que cada um desses tópicos tivesse o seu próprio capítulo. Isso facilita a abordagem de um único tópico, como a segurança da informação, como parte de um curso mais geral. Adicionei um capítulo completamente novo sobre engenharia de resiliência, que aborda cibersegurança, resiliência organizacional e projeto de sistemas resilientes.
3. Na Parte 3, adicionei novos capítulos sobre engenharia de sistemas e sobre os sistemas de sistemas, além de ter revisado extensivamente o material sobre engenharia de sistemas orientados a serviços, para refletir o uso crescente de serviços RESTful. O capítulo sobre engenharia de software orientada a aspectos foi excluído da versão impressa, mas permanece disponível como um capítulo no site do livro (em inglês).

4. Na Parte 4, atualizei o material sobre gerenciamento de configuração para refletir o uso crescente de ferramentas de controle de versão distribuídas, como Git. O capítulo sobre melhoria de processos foi excluído da versão impressa, mas permanece disponível como um capítulo no site do livro (em inglês).

Uma mudança importante no material suplementar para o livro é a adição de recomendações de vídeo em todos os capítulos. Eu fiz mais de 40 vídeos sobre uma série de tópicos que estão disponíveis no meu canal do YouTube e cujos *links* se encontram nas páginas do site do livro. Nos casos em que não fiz vídeos, recomendei vídeos do YouTube que podem ser úteis.

PÚBLICO

O livro destina-se principalmente a estudantes universitários que frequentam cursos introdutórios e avançados de engenharia de software e sistemas. Presumo que os leitores compreendam as noções básicas de programação e as estruturas de dados fundamentais.

Engenheiros de software na indústria podem considerar este livro útil como leitura geral e para atualizar seus conhecimentos sobre temas como reúso de software, projeto de arquitetura de sistemas, dependabilidade (*dependability*) e segurança da informação (*security*), além de engenharia de sistemas.

USANDO O LIVRO EM CURSOS DE ENGENHARIA DE SOFTWARE

Projetei o livro para que ele possa ser usado em três tipos diferentes de curso de engenharia de software:

1. *Cursos introdutórios gerais de engenharia de software.* A primeira parte do livro foi projetada para apoiar um curso de um semestre de introdução à engenharia de software. Há nove capítulos que abrangem temas fundamentais em engenharia de software. Se o seu curso tiver um componente prático, os capítulos de gestão da Parte 4 podem ser substituídos por alguns destes.
2. *Cursos introdutórios ou intermediários em tópicos específicos de engenharia de software.* Você pode criar uma série de cursos mais avançados usando os capítulos das Partes 2-4. Por exemplo, ministrei um curso sobre sistemas críticos usando os capítulos da Parte 2, mais os capítulos sobre engenharia de sistemas e gerenciamento da qualidade. Em um curso que abrange a engenharia de sistemas intensivos de software, usei capítulos sobre engenharia de sistemas, engenharia de requisitos, sistemas de sistemas, engenharia de software distribuído, software embarcado, gerenciamento de projetos e planejamento de projetos.
3. *Cursos mais avançados em tópicos específicos de engenharia de software.* Neste caso, os capítulos do livro formam uma base para o curso. Estes são complementados com uma leitura mais aprofundada que explora o tema em mais detalhes. Por exemplo, um curso sobre reúso de software pode ter como base os Capítulos 15–18.

SITE DO LIVRO

Este livro foi projetado como um texto híbrido impresso/digital em que a informação fundamental na edição impressa está vinculada ao material suplementar (em inglês) na internet. Vários capítulos incluem ‘seções web’ especialmente escritas que adicionam informações a cada capítulo. Essas seções incluem *QR codes* para o conteúdo do site que podem ser ativados diretamente de um smartphone. Há também seis capítulos no site sobre temas que não abordei na versão impressa do livro.

Você pode baixar uma ampla gama de material de apoio, em inglês, do site do livro (software-engineering-book.com), incluindo:

- ▶ Apresentações em PowerPoint para todos os capítulos.
- ▶ Um conjunto de vídeos nos quais abordo uma série de tópicos de engenharia de software. Também recomendo outros vídeos do YouTube que podem apoiar a aprendizagem.
- ▶ Um guia do instrutor com conselhos sobre como usar o livro para lecionar em diferentes cursos.
- ▶ Mais informações sobre estudos de caso do livro (bomba de insulina, sistema de informação de pacientes para cuidados com a saúde mental, sistema meteorológico na natureza, sistema digital de aprendizagem), bem como outros estudos de caso, como a falha do lançamento do Ariane 5.
- ▶ Seis capítulos web que abordam melhoria de processos, métodos formais, projeto de interação, arquiteturas de aplicação, documentação e desenvolvimento orientado a aspectos.
- ▶ Seções web que se somam ao conteúdo apresentado em cada capítulo. Essas seções web estão vinculadas a *boxes* eventuais incluídos em cada capítulo.
- ▶ Outras apresentações em PowerPoint que abordam uma gama de tópicos de engenharia de sistemas.

Em resposta aos pedidos de leitores do livro, publiquei uma especificação de requisitos completa para um dos sistemas dos estudos de caso no site do livro. É difícil que alunos tenham acesso a esse tipo de documento e, portanto, entendam a sua arquitetura e complexidade. Para evitar problemas de confidencialidade, reformulei o documento de requisitos de um sistema real para que não haja restrições ao seu uso.

DETALHES DE CONTATO

Website: software-engineering-book.com

E-mail: software.engineering.book@gmail.com

Blog: iansommerville.com/systems-software-and-technology

YouTube: youtube.com/user/SoftwareEngBook

Facebook: facebook.com/sommerville.software.engineering

Twitter: @SoftwareEngBook ou @iansommerville (para *tweets* mais gerais)

Siga-me no Twitter ou no Facebook para obter atualizações sobre novos materiais e comentários sobre engenharia de software e sistemas.

AGRADECIMENTOS

Muitas pessoas contribuíram ao longo dos anos para a evolução deste livro e eu gostaria de agradecer a todas elas (revisores, estudantes e leitores) que fizeram comentários sobre edições anteriores e sugestões construtivas para modificações. Eu particularmente gostaria de agradecer a minha família, Anne, Ali e Jane, por seu amor, ajuda e apoio, enquanto eu estava trabalhando neste livro (e em todas as edições anteriores).

Ian Sommerville

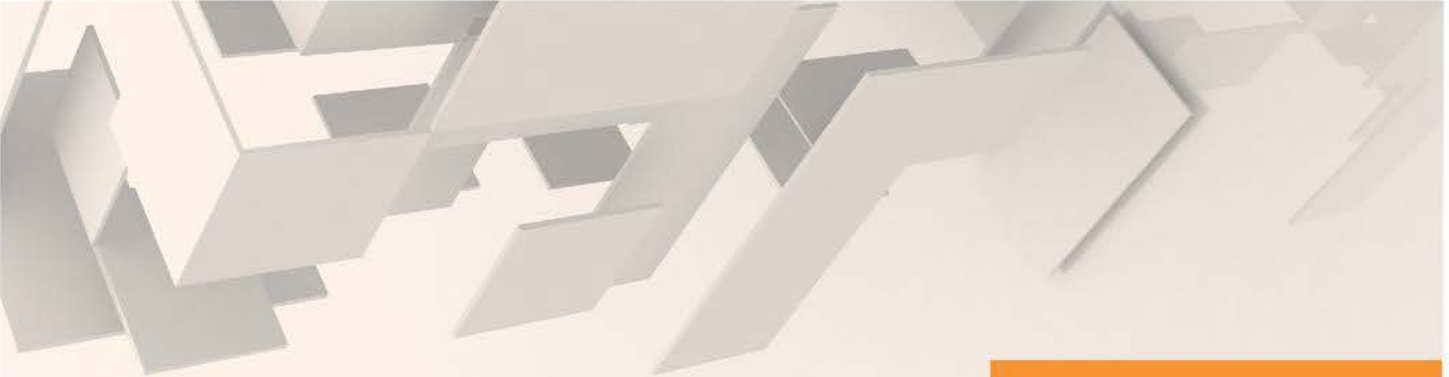
MATERIAL DE APOIO



Na Sala Virtual deste livro (sv.pearson.com.br), professores podem acessar os seguintes materiais adicionais:

- ▶ Apresentações em PowerPoint
- ▶ Galeria de imagens
- ▶ Manual de soluções (em inglês)
- ▶ Exercícios adicionais (em inglês)

Esse material é de uso exclusivo para professores e está protegido por senha. Para solicitar o acesso, os professores que adotam o livro devem entrar em contato com seu representante Pearson ou enviar e-mail para professoraluno.atendimento@pearson.com.



Parte 1

Introdução à engenharia de software

Meu objetivo nesta parte do livro é fornecer uma introdução geral à engenharia de software; os capítulos foram concebidos para um primeiro curso, com um semestre de duração. Apresento conceitos importantes, como os processos de software e os métodos ágeis, e descrevo atividades essenciais do desenvolvimento de software: da especificação dos requisitos até a evolução do sistema.

O Capítulo 1 é uma apresentação à engenharia de software profissional e traz a definição de alguns conceitos da área. Também inclui uma breve discussão sobre questões éticas como, por exemplo, a importância de os engenheiros de software pensarem a respeito das implicações mais amplas de seu trabalho. Esse capítulo também apresenta quatro estudos de caso que utilizo no livro: um sistema de informações para gerenciar registros de pacientes submetidos a tratamento de problemas de saúde mental (Mentcare), um sistema de controle para uma bomba de insulina portátil, um sistema embarcado em uma estação meteorológica na natureza e um ambiente digital de aprendizagem (iLearn).

Os dois capítulos seguintes cobrem os processos de engenharia de software e o desenvolvimento ágil. No Capítulo 2, apresento os modelos de processo, como o modelo em cascata, e discuto suas atividades básicas. O Capítulo 3 complementa o assunto com uma discussão sobre métodos de desenvolvimento ágil na engenharia de software. Esse capítulo foi amplamente alterado em relação às edições anteriores, com um foco no desenvolvimento ágil usando Scrum e uma discussão sobre as práticas ágeis, como as histórias utilizadas na definição dos requisitos e o desenvolvimento dirigido por testes (*test-driven development*).

Os próximos capítulos desta parte são descrições ampliadas das atividades de processo de software apresentadas no Capítulo 2. O Capítulo 4 cobre o tópico extremamente importante da engenharia de requisitos, que define os requisitos do que o sistema deve fazer. O Capítulo 5 explica a modelagem de sistemas usando a UML, e me concentro no uso de diagramas de caso de uso, diagramas de classe, diagramas de sequência e diagramas de estado para modelar um sistema de software. No Capítulo 6, discuto a importância da arquitetura de software e o uso de padrões de arquitetura no projeto de software.

O Capítulo 7 apresenta o projeto orientado a objetos e o uso de padrões de projeto (*design patterns*). Também apresento questões importantes de implementação — reúso, gerenciamento de configuração e desenvolvimento *host-target* — e discuto o desenvolvimento de código aberto (*open source*). O Capítulo 8 se concentra no teste de software, desde o teste de unidade, durante o desenvolvimento do sistema, até o teste de versões de software. Também discuto o uso do desenvolvimento dirigido por testes — uma abordagem pioneira nos métodos ágeis, mas que tem ampla aplicabilidade. Finalmente, o Capítulo 9 apresenta uma visão geral das questões de evolução do software, em que cubro os processos de evolução, a manutenção de software e o gerenciamento de sistemas legados.

1

Introdução

OBJETIVOS

Os objetivos deste capítulo são introduzir a engenharia de software e fornecer uma estrutura para a compreensão do conteúdo do livro. Ao ler este capítulo, você:

- ▶ compreenderá o que é a engenharia de software e porque ela é importante;
- ▶ compreenderá que o desenvolvimento dos diferentes tipos de sistema de software pode exigir diferentes técnicas de engenharia;
- ▶ compreenderá questões éticas e profissionais importantes para os engenheiros de software;
- ▶ será apresentado a quatro sistemas, de diferentes tipos, que são utilizados como exemplos ao longo deste livro.

CONTEÚDO

- 1.1 Desenvolvimento profissional de software
- 1.2 Ética da engenharia de software
- 1.3 Estudos de caso

A engenharia de software é essencial para o funcionamento do governo, da sociedade e de empresas e instituições nacionais e internacionais. O mundo moderno não funciona sem software. A infraestrutura e os serviços públicos nacionais são controlados por sistemas computacionais e a maioria dos produtos elétricos inclui um computador e um software que o controla. A produção e distribuição industriais são completamente informatizadas, assim como o sistema financeiro. O setor de entretenimento — incluindo a música, os jogos de computador, o cinema e a televisão —, usa software intensivamente. Mais de 75% da população mundial possui um telefone celular controlado por software; e, até 2016, quase todos eles acessarão a internet.

Os sistemas de software são abstratos e intangíveis. Eles não são limitados pelas propriedades dos materiais e nem são governados pelas leis da física ou pelos processos de produção. Isso simplifica a engenharia de software, uma vez que não há limites naturais para o potencial do software. No entanto, devido à ausência de limitações físicas, os sistemas de software podem rapidamente se tornar bastante complexos, difíceis de entender e caros de modificar.

Existem muitos tipos diferentes de sistemas de software, variando de sistemas embarcados simples até sistemas de informação complexos, de alcance mundial. Não

há notações, métodos ou técnicas universais para a engenharia de software, pois os diferentes tipos de software exigem abordagens igualmente diferentes. Desenvolver um sistema de informações organizacionais é completamente diferente de desenvolver um controlador para um instrumento científico. Nenhum deles têm muita coisa em comum com um jogo de computador que utiliza os gráficos intensivamente. Todas essas aplicações precisam de engenharia de software, mas nem todas elas precisam dos mesmos métodos e das mesmas técnicas.

Existem, ainda, muitos relatos de projetos equivocados e de “falhas de software”. A engenharia de software é criticada como inadequada para o desenvolvimento de software moderno. Entretanto, acredito que muitas dessas ditas falhas de software sejam consequência de dois fatores:

1. *Complexidade crescente dos sistemas.* À medida que as novas técnicas de engenharia de software nos ajudam a construir sistemas maiores e mais complexos, as demandas mudam. Os sistemas precisam ser construídos e distribuídos com mais rapidez; é necessário que sejam ainda maiores e mais complexos; e eles precisam ter novas capacidades, que antes eram consideradas impossíveis. Novas técnicas de engenharia de software precisam ser desenvolvidas para enfrentar os novos desafios de fornecer um software mais complexo.
2. *Não utilização de métodos de engenharia de software.* É bem fácil escrever programas de computador sem usar métodos e técnicas de engenharia de software. Muitas empresas se aventuraram no desenvolvimento de software conforme seus produtos e serviços evoluíram. Como não usam métodos de engenharia de software em seu cotidiano, seu software costuma ser mais caro e menos confiável do que deveria. Precisamos de mais formação e treinamento em engenharia de software para solucionar esse problema.

Os engenheiros de software podem se orgulhar, com razão, de suas realizações. Naturalmente, ainda temos problemas para desenvolver software complexo, mas sem a engenharia de software não teríamos explorado o espaço e nem teríamos a internet ou as telecomunicações modernas. Todas as formas de viagem seriam mais perigosas e caras. Os desafios para a humanidade no século XXI são a mudança climática, a menor disponibilidade de recursos naturais, a variação demográfica e uma população mundial crescente. Contaremos com a engenharia de software para desenvolver os sistemas de que precisamos para lidar com essas questões.

História da engenharia de software

O conceito de engenharia de software foi proposto pela primeira vez em 1968, em uma conferência realizada para discutir o que então se chamava crise do software (NAUR; RANDELL, 1969). Ficou claro que as abordagens individuais ao desenvolvimento de programas não escalavam para sistemas de software grandes e complexos. Os sistemas não eram confiáveis, custavam mais do que o previsto e eram entregues com atraso.

Durante os anos 1970 e 1980, foi desenvolvida uma série de técnicas e métodos de engenharia de software, como a programação estruturada, a ocultação da informação (*information hiding*) e o desenvolvimento orientado a objetos. Foram desenvolvidas ferramentas e notações que compõem a base da engenharia de software atual.



1.1 DESENVOLVIMENTO PROFISSIONAL DE SOFTWARE

Muita gente escreve programas de computador. Nas empresas, as pessoas escrevem programas de planilha eletrônica para simplificar seus trabalhos; cientistas e engenheiros escrevem programas para processar seus dados experimentais; amadores escrevem programas para seu próprio interesse e prazer, por *hobby*. No entanto, a maior parte do desenvolvimento de software é uma atividade profissional em que o software é desenvolvido para fins comerciais, para inclusão em outros dispositivos ou como produto de software, a exemplo dos sistemas de informação e dos sistemas de projeto assistido por computador. As distinções fundamentais são que o software profissional se destina a ser utilizado por outras pessoas além de seu desenvolvedor e que ele é desenvolvido normalmente por times, e não por indivíduos; ele é mantido e alterado ao longo de sua vida útil.

A engenharia de software se destina a apoiar o desenvolvimento de software profissional em vez de a programação individual. Ela inclui técnicas que apoiam a especificação, o projeto e a evolução do software, aspectos geralmente irrelevantes para o desenvolvimento de software pessoal. Para ajudar a ter uma visão geral da engenharia de software, resumi na Figura 1.1 as perguntas mais frequentes sobre o assunto.

FIGURA 1.1 Perguntas frequentes sobre engenharia de software.

Pergunta	Resposta
O que é software?	Programas de computador e documentação associada. Os produtos de software podem ser desenvolvidos para um determinado cliente ou para um mercado genérico.
Quais são os atributos do bom software?	O bom software deve proporcionar a funcionalidade e o desempenho necessários e deve ser manutenível, usável e com dependabilidade (<i>dependability</i>).
O que é engenharia de software?	A engenharia de software é uma disciplina de engenharia que se preocupa com os aspectos da produção de software, desde sua concepção inicial até sua operação e manutenção.
Quais são as atividades fundamentais da engenharia de software?	Especificação, desenvolvimento, validação e evolução do software.
Qual é a diferença entre engenharia de software e ciência da computação?	A ciência da computação se concentra na teoria e nos fundamentos. A engenharia de software se preocupa com as questões práticas de desenvolver e entregar software útil.
Qual é a diferença entre engenharia de software e engenharia de sistemas?	A engenharia de sistemas se preocupa com todos os aspectos do desenvolvimento de sistemas computacionais, incluindo hardware, software e engenharia de processos. A engenharia de software faz parte desse processo mais geral.
Quais são os principais desafios enfrentados pela engenharia de software?	Lidar com a crescente diversidade, com as demandas por menores prazos de entrega e desenvolver software confiável.
Quais são os custos da engenharia de software?	Aproximadamente 60% dos custos de software são relativos ao desenvolvimento e 40%, aos testes. Quanto ao software personalizado, os custos de evolução frequentemente ultrapassam os de desenvolvimento.
Quais são os melhores métodos e técnicas de engenharia de software?	Ainda que todos os projetos de software devam ser gerenciados e desenvolvidos profissionalmente, técnicas diferentes são adequadas para tipos diferentes de sistemas. Por exemplo, jogos devem ser sempre desenvolvidos usando uma série de protótipos, enquanto sistemas de controle críticos em segurança requerem o desenvolvimento de uma especificação completa e analisável. Não há métodos ou técnicas que sejam bons para todos os casos.
Quais diferenças a internet trouxe para a engenharia de software?	A internet não só levou ao desenvolvimento de sistemas massivos, largamente distribuídos, baseados em serviços, como também deu base para a criação de uma indústria de aplicativos (ou “apps”) para dispositivos móveis que mudou a economia de software.

Muitas pessoas acham que software é apenas mais um sinônimo para programa de computador. No entanto, quando falamos sobre engenharia de software, não estamos falando apenas dos programas em si, mas também de toda a documentação, bibliotecas, websites de apoio e dados de configuração associados — elementos necessários para que esses programas sejam úteis. Um sistema de software desenvolvido profissionalmente muitas vezes é mais do que um único programa. Ele pode consistir em diversos programas diferentes e em arquivos de configuração, que são utilizados para parametrizar esses programas. Também pode incluir uma documentação de sistema, que descreve sua estrutura; manual do usuário, que explica como utilizar o sistema; e sites, para os usuários baixarem informações recentes sobre o produto.

Esta é uma das diferenças mais importantes entre o desenvolvimento de software profissional e o amador. Se estiver escrevendo um programa para si próprio, que ninguém mais vai utilizar, não é necessário se preocupar em escrever manuais de programa, em documentar o projeto e assim por diante. No entanto, se estiver desenvolvendo um software que outras pessoas utilizarão e no qual outros engenheiros farão alterações, então provavelmente será preciso fornecer informações adicionais sobre o programa, assim como seu código.

Os engenheiros de software se preocupam em desenvolver produtos de software, ou seja, software que possa ser vendido para clientes. Existem dois tipos de produto de software:

1. *Produtos genéricos.* São sistemas *stand-alone* produzidos por uma organização de desenvolvimento de software e vendidos no mercado para qualquer cliente que queira comprá-los. Os exemplos desse tipo de produto incluem aplicativos para dispositivos móveis e software para PCs — como bancos de dados, processadores de texto, pacotes de desenho e ferramentas de gerenciamento de projetos. Esse tipo de software também inclui aplicações “verticais”, projetadas para um mercado específico, como sistemas de informação de bibliotecas, sistemas contábeis ou sistemas para manter registros odontológicos.
2. *Software personalizado (ou feito sob medida).* São sistemas encomendados e desenvolvidos para um determinado cliente. Uma empresa de software projeta e implementa o software especialmente para o cliente. Exemplos desse tipo de software incluem sistemas de controle para dispositivos eletrônicos, sistemas escritos para apoiar processos de negócios específicos e sistemas de controle de tráfego aéreo.

A distinção fundamental entre esses tipos de software é que, nos produtos genéricos, a organização que desenvolve o software controla a sua especificação. Isso significa que se essa organização enfrentar problemas de desenvolvimento, ela pode repensar o que está sendo desenvolvido. Nos produtos personalizados, a especificação é desenvolvida e controlada pela organização que está comprando o software. Os desenvolvedores, então, devem trabalhar com essa especificação.

No entanto, a distinção entre esses tipos de produto está ficando cada vez mais confusa. Mais e mais sistemas estão sendo criados tendo como base um produto genérico, que depois é adaptado para atender às necessidades de um determinado cliente. Os sistemas ERP (*Enterprise Resource Planning*, em português, planejamento de recursos empresariais), como os da SAP e da Oracle, são os melhores exemplos dessa abordagem: um sistema grande e complexo é adaptado para uma empresa, incorporando informações sobre as regras e os processos de negócio, relatórios necessários e assim por diante.

Quando falamos sobre a qualidade do software profissional, temos de considerar que o software é utilizado e modificado por outras pessoas além dos seus desenvolvedores. Portanto, a qualidade não tem a ver apenas com o que o software faz. Preferencialmente, ela deve incluir o comportamento do software enquanto ele está em execução, a estrutura e a organização dos programas do sistema e a documentação associada. Isso se reflete nos atributos da qualidade — também chamados de não funcionais — do software. Exemplos desses atributos são o tempo de resposta do software para uma consulta do usuário e a inteligibilidade do código do programa.

O conjunto específico de atributos que se poderia esperar de um sistema de software depende, obviamente, de sua aplicação: um sistema de controle de aeronaves deve ser seguro, um jogo interativo deve ser responsivo, um sistema de comutação telefônica deve ser confiável e assim por diante. Esses sistemas podem ser generalizados no conjunto de atributos exibido na Figura 1.2, que eu considero serem as características essenciais de um sistema de software profissional.

FIGURA 1.2 Atributos essenciais do bom software.

Característica do produto	Descrição
Aceitabilidade	O software deve ser aceitável para o tipo de usuário para o qual é projetado. Isso significa que ele deve ser inteligível, útil e compatível com os outros sistemas utilizados pelos usuários.
Dependabilidade e segurança da informação (<i>security</i>)	A dependabilidade do software inclui uma gama de características, incluindo confiabilidade, segurança da informação (<i>security</i>) e segurança (<i>safety</i>). O software com dependabilidade não deve causar danos físicos ou econômicos em caso de falha do sistema. Ele também deve ser protegido para que usuários maliciosos não consigam acessar ou danificar o sistema.
Eficiência	O software não deve desperdiçar recursos do sistema, como a memória e os ciclos de processador. Portanto, a eficiência inclui responsividade, tempo de processamento, utilização de recursos etc.
Manutenibilidade	O software deve ser escrito de tal modo que possa evoluir e satisfazer as necessidades mutáveis dos clientes. Este é um atributo crítico, pois a modificação do software é um requisito inevitável de um ambiente empresarial mutável.

1.1.1 Engenharia de software

A engenharia de software é uma disciplina relacionada a todos os aspectos da produção de software, desde os estágios iniciais da especificação, até a manutenção depois que o sistema passa a ser usado. Nessa definição, há duas frases-chave:

1. *Disciplina de engenharia.* Os engenheiros fazem as coisas funcionarem. Eles aplicam teorias, métodos e ferramentas onde for apropriado. No entanto, eles utilizam esses recursos seletivamente e sempre tentam descobrir soluções para os problemas, mesmo quando não há teorias e métodos aplicáveis. Os engenheiros também reconhecem que devem trabalhar dentro de limites organizacionais e financeiros e que devem buscar soluções dentro desses limites.

2. *Todos os aspectos da produção de software.* A engenharia de software não se preocupa apenas com os processos técnicos do desenvolvimento de software. Ela também inclui atividades como o gerenciamento de projetos de software e o desenvolvimento de ferramentas, métodos e teorias que apoiam a criação de software.

Engenharia se trata de obter resultados que atendam à qualidade exigida, dentro do cronograma e do orçamento. Muitas vezes isso envolve fazer concessões — os engenheiros não podem ser perfeccionistas. As pessoas que escrevem programas para si mesmas podem, por outro lado, investir o tempo que quiserem no desenvolvimento desses programas.

Em geral, os engenheiros de software adotam uma abordagem sistemática e organizada para o seu trabalho, já que muitas vezes essa é a maneira mais eficaz de produzir software de alta qualidade. Entretanto, engenharia se trata de escolher o método mais adequado para um conjunto de circunstâncias, de modo que uma abordagem mais criativa e menos formal de desenvolvimento pode ser a mais adequada para alguns tipos de software. Um processo de software mais flexível que acomode a mudança rápida é particularmente adequado para o desenvolvimento de sistemas web interativos e de aplicativos para dispositivos móveis, que requerem um conjunto de habilidades de projeto de software e de projeto gráfico.

A engenharia de software é importante por duas razões:

1. Cada vez mais os indivíduos e a sociedade dependem de sistemas de software avançados. Precisamos ser capazes de produzir sistemas confiáveis de maneira econômica e rápida.
2. Geralmente é mais barato, no longo prazo, usar métodos e técnicas de engenharia de software para sistemas de software profissionais em vez de apenas escrever programas como um projeto de programação pessoal. Não utilizar um método de engenharia de software leva a custos mais altos de teste, garantia de qualidade e manutenção de longo prazo.

A abordagem sistemática utilizada na engenharia de software é, às vezes, chamada de processo de software, uma sequência de atividades que leva à produção de um software. Quatro atividades fundamentais são comuns a todos os processos de software:

1. Especificação do software, etapa em que clientes e engenheiros definem o software que deve ser produzido e as restrições impostas à sua operação.
2. Desenvolvimento de software, etapa em que o software é projetado e programado.
3. Validação de software, etapa em que o programa é analisado para garantir que seja aquilo de que o cliente precisa.
4. Evolução do software, etapa de modificação para refletir a mudança de requisitos tanto do cliente quanto do mercado.

Diferentes tipos de sistemas precisam de diferentes processos de desenvolvimento, como explicarei no Capítulo 2. Por exemplo, um software de tempo real de uma aeronave deve ser completamente especificado antes de começar a ser desenvolvido. Nos sistemas de comércio eletrônico, a especificação e o programa normalmente são desenvolvidos em conjunto. Consequentemente, essas atividades genéricas podem ser organizadas de diferentes maneiras e descritas em diferentes níveis de detalhe, dependendo do tipo de software a ser desenvolvido.

A engenharia de software está relacionada tanto à ciência da computação quanto à engenharia de sistemas:

1. A ciência da computação trata das teorias e dos métodos que dão base à computação e aos sistemas de software, ao passo que a engenharia de software se preocupa com os problemas práticos da produção de software. Algum conhecimento de ciência da computação é essencial para os engenheiros de software, assim como algum conhecimento de física é essencial para os engenheiros elétricos. Nem sempre as elegantes teorias de ciência da computação são relevantes para os problemas grandes e complexos que exigem uma solução de software.
2. A engenharia de sistemas se concentra em todos os aspectos do desenvolvimento e evolução de sistemas complexos em que o software desempenha um papel importante. Assim, a engenharia de sistemas se preocupa com o desenvolvimento do hardware, com o projeto das políticas e processos e com a implantação de sistemas, assim como com a engenharia de software. Os engenheiros de sistemas se envolvem na especificação do sistema, na definição de sua arquitetura global e, depois, em integrar as diferentes partes para criar o sistema acabado.

Conforme veremos na próxima seção, existem muitos tipos diferentes de software. Não existem métodos ou técnicas universais de engenharia de software que possam ser utilizados. No entanto, existem quatro questões relacionadas que afetam muitos tipos de software diferentes:

1. *Heterogeneidade.* Cada vez mais são necessários sistemas que atuem de forma distribuída em redes e que incluam diferentes tipos de computadores e dispositivos móveis. Assim como nos computadores de uso geral, o software também pode precisar ser executado em telefones celulares e tablets. Muitas vezes é necessário integrar um novo software com sistemas legados mais antigos escritos em diferentes linguagens de programação. O desafio aqui é o de desenvolver técnicas para criar software com dependabilidade que seja suficientemente flexível para lidar com essa heterogeneidade.
2. *Mudanças nos negócios e na sociedade.* As empresas e a sociedade estão mudando com uma rapidez incrível à medida que economias emergentes se desenvolvem e novas tecnologias são disponibilizadas. Elas precisam ser capazes de mudar seu software existente e de desenvolver rapidamente um novo. Muitas técnicas tradicionais de engenharia de software são demoradas, e a entrega dos novos sistemas frequentemente leva mais tempo que o planejado. Essas técnicas precisam evoluir, de modo que o tempo necessário para que o software entregue valor aos seus clientes seja reduzido.
3. *Segurança da informação (security) e confiança (trust).* Pelo fato de o software estar entranhado em todos os aspectos de nossas vidas, é essencial que possamos confiar nele. Isso vale especialmente para os sistemas de software remoto acessados por meio de uma página da internet ou de uma interface de serviços web. Precisamos ter certeza de que usuários maliciosos não consigam atacar com sucesso o nosso software e que as informações estejam protegidas.
4. *Escala.* O software tem de ser desenvolvido considerando diferentes escalas, desde pequenos sistemas embarcados em dispositivos móveis ou dispositivos

vestíveis (também chamados de *wearables*) até sistemas baseados na nuvem e distribuídos pela internet que atendam a uma comunidade global.

Para enfrentar esses desafios, precisaremos de novas ferramentas e técnicas, além de maneiras inovadoras de combinar e usar métodos de engenharia de software existentes.

1.1.2 Diversidade da engenharia de software

A engenharia de software é uma abordagem sistemática para a produção de software que leva em conta o custo prático, o cronograma e questões de dependabilidade, bem como as necessidades de clientes e produtores de software. Métodos, ferramentas e técnicas específicas dependem da organização que está desenvolvendo o software, do tipo de software e das pessoas envolvidas no processo de desenvolvimento. Não há métodos universais de engenharia de software que sejam adequados para todos os sistemas e empresas. Em vez disso, um conjunto diverso de métodos e ferramentas de engenharia de software evoluiu ao longo dos últimos 50 anos. Entretanto, a iniciativa SEMAT (JACOBSON *et al.*, 2013) propõe que haja um metaproceto fundamental instanciado para criar diferentes tipos de processos. Esse é um estágio inicial do desenvolvimento e pode ser uma base para melhorar os nossos atuais métodos de engenharia de software.

Talvez o fator mais importante na determinação de quais são os métodos e técnicas de engenharia de software mais importantes é o tipo de aplicação que está sendo desenvolvido. Existem muitos tipos diferentes de aplicação, incluindo:

1. *Aplicações stand-alone.* São sistemas de aplicação executados em um computador pessoal ou aplicativos que rodam em dispositivos móveis. Elas incluem toda a funcionalidade necessária e podem não necessitar de conexão a uma rede. Como exemplos, temos as aplicações de escritório em um computador pessoal, programas de CAD, software de manipulação de imagens, aplicativos de viagem, aplicativos de produtividade etc.
2. *Aplicações interativas baseadas em transações.* São aplicações executadas em um computador remoto e que são acessadas por usuários a partir de seus próprios computadores, smartphones ou tablets. Elas incluem aplicações web como as de comércio eletrônico, por exemplo, por meio das quais se interage com um sistema remoto para comprar bens e serviços. Essa classe de aplicação também inclui sistemas de negócio, nos quais uma empresa concede acesso a seus sistemas por meio de um navegador, de um programa cliente de uso específico ou de um serviço baseado na nuvem, como e-mail e compartilhamento de imagens. As aplicações interativas incorporam frequentemente um grande armazenamento de dados que é acessado e atualizado em cada transação.
3. *Sistemas de controle embarcados.* São sistemas de controle de software que controlam e gerenciam dispositivos de hardware. Em números, existem provavelmente mais sistemas embarcados do que qualquer outro tipo. Exemplos de sistemas embarcados incluem o software de um telefone celular, o software que controla o freio ABS em um carro e o software em um forno de micro-ondas para controlar o processo de cozimento.

4. *Sistemas de processamento em lotes (batch)*. São sistemas de negócio concebidos para processar dados em grandes lotes. Eles processam números enormes de entradas individuais para criar as saídas correspondentes. Exemplos de sistemas em lote incluem o faturamento periódico, como as contas de telefone, e os sistemas de folha de pagamento.
5. *Sistemas de entretenimento*. São destinados para uso pessoal, para entreter o usuário. A maioria desses sistemas consiste em jogos de gêneros variados, que podem ser executados em um console concebido especificamente para essa finalidade. A qualidade da interação com o usuário é a característica diferenciadora mais importante dos sistemas de entretenimento.
6. *Sistemas para modelagem e simulação*. São desenvolvidos por cientistas e engenheiros para modelar processos físicos ou situações que incluem muitos objetos diferentes e que interagem. Costumam ser computacionalmente intensivos e demandam sistemas paralelos de alto desempenho para a sua execução.
7. *Sistemas de coleta de dados e análise*. São aqueles que fazem a sua coleta no ambiente e enviam esses dados para outros sistemas, para processamento. O software pode ter de interagir com sensores e frequentemente é instalado em um ambiente hostil, como o interior de um motor, ou em uma localização remota. A análise de 'Big Data' (grandes volumes de dados) pode envolver sistemas baseados na nuvem executando análises estatísticas e procurando relações entre os dados coletados.
8. *Sistemas de sistemas*. São utilizados em empresas e outras grandes organizações e são compostos de uma série de outros sistemas de software. Alguns deles podem ser produtos de software genéricos, como um sistema ERP. Outros sistemas do conjunto podem ser desenvolvidos especialmente para esse ambiente.

Naturalmente, as fronteiras entre esses tipos de sistema são tênues. Ao desenvolver um jogo para um telefone celular, deve-se levar em conta as mesmas restrições (potência, interação com o hardware) que os desenvolvedores do software do telefone. Os sistemas de processamento em lotes são utilizados frequentemente em conjunto com os sistemas de transações baseados na web. Em uma empresa, por exemplo, os créditos pelas despesas de viagem podem ser solicitados através de uma aplicação web, mas processados em uma aplicação em lotes (*batch*) para pagamento mensal.

Cada tipo de sistema exige técnicas especializadas de engenharia de software, pois cada um tem características diferentes. Por exemplo, um sistema de controle embarcado em um automóvel é um sistema crítico em segurança¹ e é gravado em uma ROM (memória somente de leitura, do inglês *Read Only Memory*) ao ser instalado no veículo. Por isso, sua modificação é muito cara. Esse tipo de sistema precisa de verificação e validação amplas, a fim de que a probabilidade de os carros sofrerem um *recall* após a venda seja minimizada. A interação com o usuário é mínima (ou talvez inexistente), então não há necessidade de usar um processo de desenvolvimento que se baseie na prototipação da interface com o usuário.

Para um sistema ou aplicativo web, a melhor abordagem é o desenvolvimento e a entrega iterativos, em que o sistema é composto de componentes reusáveis. No entanto, essa abordagem pode ser impraticável para um sistema de sistemas, no qual as especificações detalhadas das interações do sistema têm de ser especificadas de antemão para que cada sistema seja desenvolvido separadamente.

¹ N. da R.T.: *safety-critical system* é às vezes traduzido como 'sistema de segurança crítica'.

Apesar disso, há fundamentos da engenharia de software que se aplicam a todos os tipos de sistemas de software:

1. Eles devem ser desenvolvidos com o uso de um processo gerenciado e compreendido. A organização que está desenvolvendo o software deve planejar o processo de desenvolvimento e ter ideias claras do que será produzido e de quando será concluído. Naturalmente, o processo específico que deve ser utilizado depende do tipo de software que se está desenvolvendo.
2. Dependabilidade e desempenho são importantes para todos os tipos de sistema. O software deve se comportar conforme o esperado, sem falhas, e deve estar disponível para uso quando for necessário. Deve ter uma operação segura e, na medida do possível, ter proteção contra ataques externos. O sistema deve ter um desempenho eficiente e não desperdiçar recursos.
3. É importante compreender e controlar a especificação e os requisitos do software (o que o software deve fazer). É preciso saber o que os diferentes clientes e usuários esperam do sistema e gerenciar as expectativas deles a fim de fornecer um sistema útil dentro do orçamento e do cronograma.
4. Os recursos existentes devem ser usados de modo eficaz. Isso significa que, onde for apropriado, deve-se reusar software que já tenha sido desenvolvido, em vez de escrever um novo.

Esses conceitos fundamentais de processo, dependabilidade, requisitos, gerenciamento e reúso são temas importantes deste livro. Diferentes métodos os refletem de diferentes maneiras, mas eles estão na base de todo o desenvolvimento de software profissional.

Esses fundamentos independem da linguagem de programação utilizada no desenvolvimento do software. Não abordo técnicas de programação específicas neste livro porque elas variam radicalmente de um tipo de sistema para outro. Por exemplo, uma linguagem dinâmica, como Ruby, é o tipo correto de linguagem para o desenvolvimento de sistemas interativos, mas é inadequado para a engenharia de sistemas embarcados.

1.1.3 Engenharia de software para internet

O desenvolvimento da internet e da rede mundial de computadores tem surtido um efeito profundo nas vidas de todos nós. Inicialmente, a web era um repositório de informações acessível universalmente e surtia pouco efeito nos sistemas de software. Esses sistemas eram executados em computadores locais e só eram acessíveis de dentro de uma organização. Por volta do ano 2000, a web começou a evoluir e cada vez mais recursos foram adicionados aos navegadores, indicando que os sistemas web poderiam ser desenvolvidos e que, no lugar de uma interface com o usuário específica, eles passariam a ser acessados por meio de um navegador. Isso levou ao desenvolvimento de uma ampla gama de novos produtos de sistema que ofereciam serviços inovadores acessados pela internet. Eles são frequentemente mantidos por anúncios exibidos na tela e não envolvem o pagamento direto pelos usuários.

Assim como esses produtos de sistema, o desenvolvimento de navegadores web que pudessem executar pequenos programas e fazer algum processamento local levou a uma evolução em softwares corporativos e organizacionais. Em vez de escrever um software e instalá-lo nos computadores dos usuários, ele poderia ser implantado em

um servidor web. Isso barateou a modificação e a atualização de software, já que não havia necessidade de instalá-lo em cada computador. Isso também reduziu custos, pois o desenvolvimento da interface com o usuário é particularmente caro. Sempre que foi possível fazê-lo, empresas migraram seus sistemas de software para uma interação web.

O conceito de software como serviço (Capítulo 17) foi proposto no início do século XXI. Hoje, essa é a abordagem padrão para a distribuição de produtos web como Google Apps, Microsoft Office 365 e Adobe Creative Suite. Cada vez mais, o software é executado em 'nuvens' remotas em vez de servidores locais, e é acessado por meio da internet. Uma nuvem de computação é uma quantidade enorme de sistemas computacionais interligados que é compartilhada por muitos usuários. Os usuários não compram o software, mas pagam de acordo com o quanto ele é utilizado ou recebem acesso gratuito em troca da visualização de anúncios que são exibidos na tela. Ao usar serviços baseados na web como e-mail, armazenamento ou vídeo, um sistema em nuvem está sendo utilizado.

O advento da internet levou a uma mudança radical na maneira como o software corporativo é organizado. Antes da web, as aplicações corporativas eram praticamente monolíticas, programas únicos rodando em computadores individuais ou em *clusters*. As comunicações eram locais, restritas à empresa. Agora, o software é altamente distribuído, às vezes mundialmente. As aplicações corporativas não são programadas do zero, mas envolvem o amplo reúso de componentes e programas.

Essa mudança na organização de software teve um efeito importante na engenharia de software para os sistemas web. Por exemplo:

1. O reúso de software se tornou a abordagem dominante para construir sistemas web. Durante a criação desses sistemas, é possível pensar em como montá-los a partir de componentes e de sistemas de software preexistentes, muitas vezes agrupados em um *framework*.
2. De maneira geral, hoje reconhece-se que é impraticável especificar todos os requisitos desses sistemas antecipadamente. Os sistemas web sempre são desenvolvidos e entregues de modo incremental.
3. O software pode ser implementado usando engenharia de software orientada a serviços, na qual os componentes do software são serviços da web *stand-alone*. Discutirei essa questão no Capítulo 18.
4. As tecnologias de desenvolvimento de interfaces, como AJAX (HOLDENER, 2008) e HTML5 (FREEMAN, 2011), surgiram para dar suporte à criação de interfaces dinâmicas dentro de um navegador web.

As ideias fundamentais de engenharia de software, discutidas na seção anterior, aplicam-se ao software web da mesma forma que a outros tipos de software. Os sistemas web estão ficando cada vez maiores, então as técnicas de engenharia de software que lidam com escala e complexidade são relevantes para esses sistemas.

1.2 ÉTICA DA ENGENHARIA DE SOFTWARE

Assim como outras disciplinas de engenharia, a engenharia de software é conduzida dentro de um arcabouço social e legal que limita a liberdade das pessoas que trabalham nesse setor. Um engenheiro de software deve aceitar que o seu trabalho envolve responsabilidades mais amplas do que a simples aplicação de habilidades

técnicas. Também deve se comportar de maneira ética e moralmente responsável se quiser ser respeitado como um engenheiro profissional.

Nem é preciso dizer que os padrões normais de honestidade e integridade devem ser mantidos. Habilidades e capacidades não devem ser usadas para agir de forma desonesta ou que venha a comprometer a reputação da profissão de engenheiro de software. No entanto, existem áreas nas quais os padrões de comportamento aceitável não são delimitados pelas leis, mas por uma noção mais tênue de responsabilidade profissional. Algumas dessas áreas são:

1. *Confidencialidade.* A confidencialidade dos seus empregadores ou clientes deve ser respeitada, independentemente de um acordo ter sido assinado ou não.
2. *Competência.* Não se deve apresentar um nível de competência que não seja verdadeiro. Trabalhos que estejam fora da sua competência não devem ser aceitos.
3. *Direitos de propriedade intelectual.* Deve-se estar a par das leis locais que regem o uso da propriedade intelectual, como patentes e direitos autorais. É necessário ter o cuidado de assegurar que a propriedade intelectual dos empregadores e clientes esteja protegida.
4. *Mau uso do computador.* Não se deve aproveitar habilidades técnicas para usar indevidamente os computadores de outras pessoas. O uso indevido do computador varia desde o relativamente trivial (jogar na máquina de um empregador) até o extremamente grave (disseminação de vírus ou outro tipo de *malware*).

As sociedades e instituições profissionais têm um papel importante no estabelecimento dos padrões éticos. Organizações como a ACM (*Association for Computing Machinery*), o IEEE (*Institute of Electrical and Electronic Engineers*) e a British Computer Society publicam um código de conduta profissional ou código de ética. Os membros dessas organizações se comprometem a seguir esse código quando se associam. Esses códigos de conduta geralmente estão relacionados com o comportamento ético fundamental.

As associações profissionais, particularmente a ACM e o IEEE, cooperaram para produzir um código de ética e prática profissional. Esse código existe na forma reduzida, exibida na Figura 1.3, e na forma completa (GOTTERBARN; MILLER; ROGERSON, 1999), que acrescenta detalhes e conteúdo à versão resumida. A lógica por trás do código está resumida nos dois primeiros parágrafos da versão completa:

Os computadores têm um papel fundamental e crescente no comércio, na indústria, no governo, na medicina, na educação, no entretenimento e na sociedade em geral. Os engenheiros de software são aqueles que contribuem para análise, especificação, projeto, desenvolvimento, certificação, manutenção e teste dos sistemas de software, participando diretamente ou ensinando. Em virtude de seus papéis no desenvolvimento dos sistemas de software, os engenheiros de software têm oportunidades significativas para fazer o bem ou provocar danos, capacitar terceiros a fazer o bem ou causar danos, ou influenciar terceiros a fazer o bem ou causar danos. Para assegurar o máximo possível que seus esforços serão utilizados para o bem, os engenheiros de software devem se comprometer a fazer da engenharia de software uma profissão útil e respeitada. De acordo com esse compromisso, os engenheiros de software devem seguir o seguinte código de ética e prática profissional.

O código contém oito princípios relacionados ao comportamento e às decisões tomadas pelos engenheiros de software profissionais, incluindo os praticantes, educadores, gestores, supervisores e criadores de políticas, bem como estagiários e alunos da profissão. Os princípios identificam as relações eticamente responsáveis em que indivíduos, grupos e organizações participam e as obrigações primárias dentro dessas relações. As cláusulas de cada princípio são exemplos de algumas obrigações incluídas nessas relações. Essas obrigações estão fundamentadas na humanidade da engenharia de software, com uma atenção especial dedicada às pessoas afetadas pelo trabalho dos engenheiros de software, e nos elementos exclusivos da prática da engenharia de software. O código prescreve essas obrigações de qualquer um que reivindique ou aspire ser um engenheiro de software.²

FIGURA 1.3 Código de ética da ACM/IEEE (© 1999 by the ACM Inc. and the IEEE, Inc.).

Código de ética e prática profissional da engenharia de software

Força-tarefa conjunta da ACM/IEEE-CS para ética e práticas profissionais da engenharia de software

Prefácio

A versão reduzida do código resume as aspirações em um alto nível de abstração; as cláusulas incluídas na versão completa fornecem exemplos e detalhes de como essas aspirações mudam o nosso modo de agir como profissionais de engenharia de software. Sem as aspirações, os detalhes podem se tornar legalistas e tediosos; sem os detalhes, as aspirações podem ficar pomposas, porém vazias; juntos, as aspirações e os detalhes formam um código coeso.

Os engenheiros de software devem se comprometer a fazer da análise, especificação, projeto, desenvolvimento, teste e manutenção do software uma profissão útil e respeitada. De acordo com o seu compromisso com a saúde, segurança e bem-estar do público, os engenheiros de software devem obedecer aos oito princípios a seguir:

1. Público — Os engenheiros de software devem agir coerentemente com o interesse público.
2. Cliente e empregador — Os engenheiros de software devem agir de uma maneira que atenda aos interesses de seu cliente e empregador, coerente com o interesse público.
3. Produto — Os engenheiros de software devem assegurar que seus produtos e modificações relacionadas cumpram o máximo possível os mais altos padrões profissionais.
4. Opinião — Os engenheiros de software devem manter a integridade e independência em sua opinião profissional.
5. Gestão — Os gestores e líderes em engenharia de software devem aceitar e promover uma abordagem ética do gerenciamento do desenvolvimento e manutenção do software.
6. Profissão — Os engenheiros de software devem promover a integridade e a reputação da profissão em conformidade com o interesse público.
7. Colegas — Os engenheiros de software devem ser justos e apoiar os colegas.
8. Caráter — Os engenheiros de software devem aderir a uma aprendizagem contínua durante toda a vida no que diz respeito à prática de sua profissão e promover uma abordagem ética para a prática da profissão.

Versão resumida. Disponível em: <<https://ethics.acm.org/code-of-ethics>>. Acesso em: 27 mar. 2018.

² Força-tarefa conjunta da ACM/IEEE-CS para ética e práticas profissionais da engenharia de software, prefácio da versão resumida. Disponível em: <<https://ethics.acm.org/code-of-ethics>>. Acesso em: 27 mar. 2018. Copyright © 1999 by the Association for Computing Machinery, Inc. and the Institute for Electrical and Electronics Engineers, Inc.

Em qualquer situação em que diferentes pessoas têm diferentes visões e objetivos, é provável se deparar com dilemas éticos. Por exemplo, se você discordar, em princípio, das políticas dos funcionários de maior escalão na empresa, como deverá reagir? Claramente, isso depende das pessoas envolvidas e da natureza da discordância. Será melhor defender seu trabalho na empresa ou se demitir? Se você sentir que existem problemas com um projeto de software, quando deverá revelá-los à gestão? Se você discutir esses problemas enquanto ainda são apenas uma suspeita, você poderá estar reagindo exageradamente a uma situação; se demorar demais, poderá ser impossível resolvê-los.

Todos nós enfrentamos dilemas éticos em nossas vidas profissionais e, felizmente, na maioria dos casos, eles são relativamente pequenos ou podem ser resolvidos sem grande dificuldade. Nas situações em que esses dilemas não podem ser solucionados, o engenheiro enfrenta, talvez, outro problema. A ação movida por princípios pode ser pedir demissão, mas isso pode afetar outras pessoas, como familiares, cônjuge ou filhos.

Uma dificuldade profissional para os engenheiros de software surge quando o empregador age de maneira antiética. Digamos que uma empresa seja responsável por desenvolver um sistema crítico em segurança e, devido à pressão do tempo, falsifique os registros de validação da segurança (*safety*). A responsabilidade do engenheiro é manter a confidencialidade, alertar o cliente ou tornar público, de alguma forma, que o sistema fornecido pode não ser seguro?

O problema aqui é que não há verdade absoluta quando se trata de segurança (*safety*). Embora o sistema possa não ter sido validado de acordo com critérios predefinidos, esses critérios podem ser rigorosos demais. O sistema pode operar com segurança por todo o seu ciclo de vida. Também é possível que, mesmo quando validado adequadamente, o sistema apresente falha e provoque um acidente. A revelação precoce dos problemas pode resultar em danos ao empregador e demais funcionários; a ocultação pode resultar em danos a terceiros.

Você deve ter a sua própria opinião sobre essas questões. Aqui, a posição ética correta depende das opiniões das pessoas envolvidas. O potencial para danos, o grau do dano e as pessoas afetadas pelos danos devem influenciar a decisão. Se a situação for muito perigosa, há justificativa para publicação dos fatos na imprensa ou nas mídias sociais. No entanto, você sempre deve tentar resolver a situação enquanto respeita os direitos do seu empregador.

Outra questão ética é a participação no desenvolvimento de sistemas militares e nucleares. Algumas pessoas têm opiniões formadas a respeito desses assuntos e preferem não participar de nenhum desenvolvimento de sistemas associados à defesa. Outras trabalharão em sistemas militares, mas não em sistemas de armamento. Contudo, outras acham que a segurança nacional é um princípio primordial e não têm objeções éticas a trabalhar em sistemas de armamento.

Nessa situação, é importante que tanto empregadores quanto funcionários exponham suas opiniões uns para os outros antecipadamente. No caso de uma organização se envolver em trabalhos militares ou nucleares, ela deve ser capaz de deixar claro que os empregados devem estar dispostos a aceitar qualquer distribuição de tarefas. Do mesmo modo, se um empregado for escolhido e deixar claro que não deseja trabalhar em tais sistemas, os empregadores não deverão pressioná-lo a participar futuramente.

A área geral da responsabilidade ética e profissional se torna cada vez mais importante à medida que os sistemas intensivos de software permeiam todos os

aspectos do trabalho e do cotidiano. Isso pode ser analisado a partir de um ponto de vista filosófico, em que os princípios básicos de ética são considerados, e a ética da engenharia de software é discutida tendo como referência esses princípios básicos. Essa é a abordagem adotada por Laudon (1995) e Johnson (2001). Textos mais recentes, como os de Tavani (2013), introduzem o conceito de *ciberética* (*cyberethics*) e abordam tanto sua fundamentação filosófica quanto as questões práticas e legais envolvidas, incluindo questões éticas para usuários de tecnologia e para desenvolvedores.

Eu acredito que uma abordagem filosófica é abstrata demais e difícil de relacionar à experiência do dia a dia, então prefiro a abordagem mais concreta incorporada aos códigos de conduta profissional (BOTT, 2005; DUQUENOY, 2007). Acho que a ética é discutida com mais propriedade no contexto da engenharia de software, e não como um assunto isolado. Portanto, não discuto a ética da engenharia de software de uma maneira abstrata, mas incluo exemplos nos exercícios que podem ser um ponto de partida para uma discussão em grupo.

1.3 ESTUDOS DE CASO

Para ilustrar os conceitos de engenharia de software, utilizo exemplos de quatro tipos de sistemas diferentes. Não utilizei apenas um estudo de caso deliberadamente, já que uma das mensagens principais deste livro é que a prática da engenharia de software depende do tipo de sistema que está sendo produzido. Escolho, portanto, um exemplo apropriado para cada conceito discutido — segurança (*safety*) e dependabilidade, modelagem de sistemas, reuso etc.

Os tipos de sistema que utilizo como estudos de caso são:

1. *Um sistema embarcado.* Esse é um sistema em que o software controla algum dispositivo de hardware e está embarcado neste dispositivo. As questões nos sistemas embarcados incluem, normalmente, o tamanho físico, a capacidade de resposta, o gerenciamento da energia etc. O exemplo de sistema embarcado que utilizo é o software para controlar uma bomba de insulina para portadores de diabetes.
2. *Um sistema de informação.* A finalidade principal desse tipo de sistema é gerenciar e proporcionar acesso a um banco de dados de informações. As questões nos sistemas de informação incluem segurança da informação (*security*), usabilidade, privacidade e manutenção da integridade dos dados. O exemplo de sistema de informação utilizado é um sistema de registros médicos.
3. *Um sistema de coleta de dados baseado em sensores.* Esse é um sistema cujas finalidades principais são coletar dados de um conjunto de sensores e processá-los de alguma maneira. Os requisitos fundamentais desses sistemas são confiabilidade, mesmo em condições ambientais hostis, e manutenibilidade. O exemplo de sistema de coleta de dados que utilizo é uma estação meteorológica na natureza.
4. *Um ambiente de suporte.* Esse é um conjunto integrado de ferramentas de software utilizadas para dar suporte a algum tipo de atividade. Ambientes de programação, como o Eclipse (VOGEL, 2012), serão o tipo de ambiente mais familiar aos leitores deste livro. Descrevo aqui um exemplo de ambiente digital de aprendizagem utilizado para suplementar a aprendizagem dos alunos nas escolas.

Cada um desses sistemas será apresentado neste capítulo; mais informações sobre cada um deles estão disponíveis no site do livro (software-engineering-book.com — em inglês).

1.3.1 Sistema de controle para bomba de insulina

A bomba de insulina é um dispositivo médico que simula a operação do pâncreas (um órgão interno). O software que controla esse sistema consiste em um sistema embarcado que coleta informações de um sensor e controla a bomba que, por sua vez, fornece uma dose controlada de insulina para o usuário.

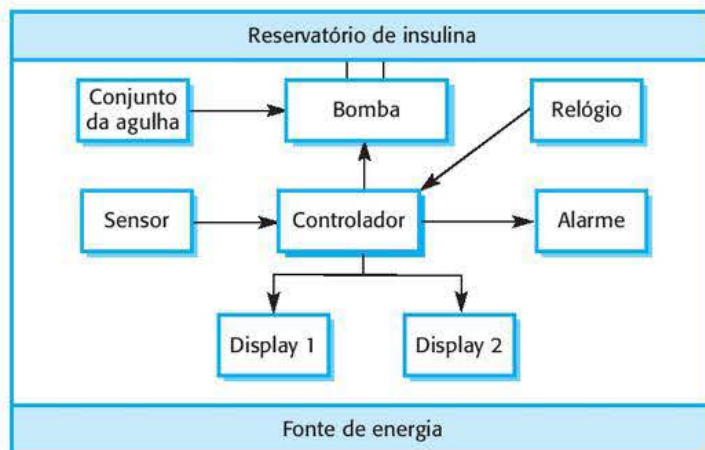
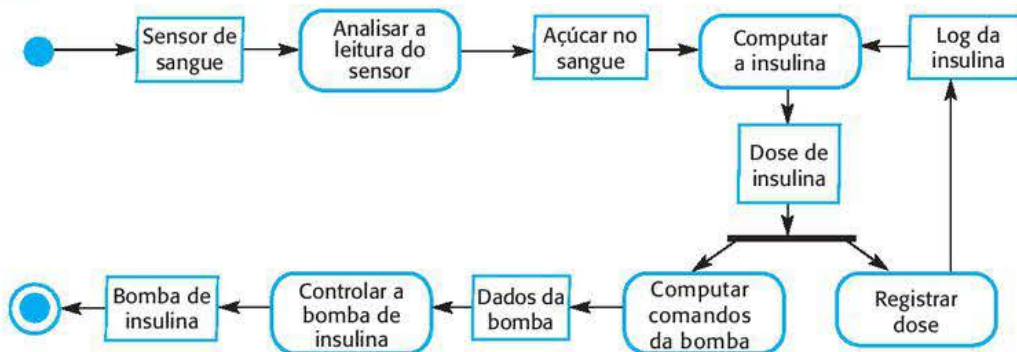
As pessoas que sofrem de diabetes usam esse sistema. Diabetes é uma condição relativamente comum na qual o pâncreas humano é incapaz de produzir quantidades suficientes de um hormônio chamado insulina, que é responsável por metabolizar a glicose (açúcar) no sangue. O tratamento convencional de diabetes envolve injeções regulares de insulina produzida através de engenharia genética. Os diabéticos medem seus níveis de glicose no sangue periodicamente usando um medidor externo e depois estimam a dose de insulina que devem injetar.

O problema é que o nível de insulina necessário não depende apenas do nível de glicose no sangue, mas também do tempo desde a última injeção. A verificação irregular pode levar a níveis muito baixos de glicose no sangue (se houver insulina demais) ou a níveis muito altos de açúcar no sangue (se houver muito pouca insulina). Glicose baixa no sangue é, no curto prazo, uma condição mais grave, já que pode resultar em mau funcionamento temporário do cérebro e, em último caso, perda de consciência e morte. Entretanto, no longo prazo, os níveis altos contínuos de glicose no sangue podem levar a danos oculares, danos renais e problemas cardíacos.

Os avanços no desenvolvimento de sensores miniaturizados tornaram possível a criação de sistemas automatizados de aplicação da insulina. Esses sistemas monitoram os níveis de açúcar no sangue e fornecem uma dose correta de insulina, quando necessária. Sistemas como esse estão disponíveis hoje em dia e são utilizados por pacientes com dificuldade para controlar os níveis de insulina. No futuro, pode ser possível que os diabéticos tenham esses sistemas acoplados permanentemente a seus corpos.

Um sistema de fornecimento de insulina controlado por software usa um microsensor embutido no paciente para medir algum parâmetro do sangue que seja proporcional ao nível de açúcar. Depois, esse parâmetro é enviado para o controlador da bomba, que computa o nível de açúcar e a quantidade de insulina necessária. Em seguida, envia sinais para uma bomba miniaturizada para fornecer a insulina através de uma agulha acoplada permanentemente.

A Figura 1.4 mostra os componentes de hardware e a organização da bomba de insulina. Para entender os exemplos apresentados neste livro, tudo o que é preciso saber é que o sensor mede a condutividade elétrica do sangue em diferentes condições e que esses valores podem estar relacionados com o nível de açúcar no sangue. A bomba de insulina fornece uma unidade de insulina em resposta a um único pulso de um controlador. Portanto, para fornecer dez unidades de insulina, o controlador envia dez pulsos para a bomba. A Figura 1.5 é um modelo de atividade em UML (*Unified Modeling Language*) que ilustra como o software transforma uma entrada do nível de açúcar no sangue em uma sequência de comandos que acionam a bomba de insulina.

FIGURA 1.4 Arquitetura do hardware de uma bomba de insulina.**FIGURA 1.5** Modelo de atividade da bomba de insulina.

Claramente, trata-se de um sistema crítico em segurança. Se a bomba não funcionar ou trabalhar de modo incorreto, a saúde do usuário pode ser prejudicada ou ele pode entrar em coma em virtude de seus níveis de açúcar no sangue estarem altos ou baixos demais. Portanto, esse sistema deve cumprir dois requisitos de alto nível essenciais:

1. O sistema deve estar disponível para fornecer insulina sempre que necessário.
2. O sistema deve funcionar de maneira confiável e fornecer a quantidade correta de insulina para compensar o nível atual de açúcar no sangue.

Assim, o sistema deve ser projetado e implementado para garantir que cumpra esses requisitos sempre. Requisitos mais detalhados e discussões de como assegurar que o sistema seja seguro serão apresentados nos próximos capítulos.

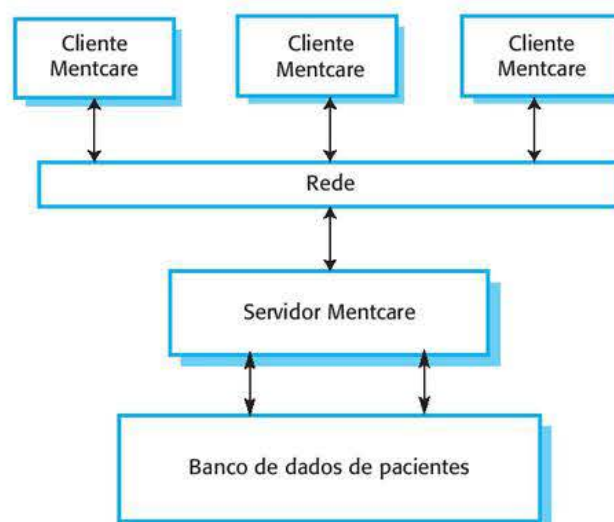
1.3.2 Sistema de informação de pacientes de saúde mental

Um sistema de informação de pacientes que fornece suporte ao cuidado com a saúde mental (o sistema Mentcare) mantém dados sobre os pacientes que sofrem de problemas de saúde mental e os tratamentos aos quais foram submetidos. A maioria dos pacientes não precisa de tratamento hospitalar dedicado, mas apenas frequentar

regularmente uma clínica especializada, onde podem encontrar um médico que tenha conhecimento detalhado de seus problemas. Para facilitar a frequência dos pacientes, essas clínicas não existem apenas dentro dos hospitais, mas em postos médicos locais ou centros comunitários.

O sistema Mentcare (Figura 1.6) é um sistema de informações de pacientes para ser utilizado em clínicas. Ele utiliza um banco de dados centralizado, que contém informações sobre os pacientes, mas também foi concebido para ser executado em um notebook, de modo que possa ser acessado e utilizado em locais sem acesso a uma conexão de rede segura. Quando os sistemas locais contam com acesso seguro à rede, eles usam as informações de pacientes que constam do banco de dados, mas podem baixar e usar cópias locais dos registros dos pacientes quando estiverem desconectados. O sistema não é um registro de informações médicas completo e, portanto, não mantém informações sobre outras condições clínicas. No entanto, ele pode interagir e trocar dados com outros sistemas de informações médicas.

FIGURA 1.6 A organização do sistema Mentcare.



Este sistema tem duas finalidades:

1. Gerar informações de gestão que permitam que gestores de serviços de saúde avaliem o desempenho em relação às metas locais e governamentais.
2. Fornecer informações atualizadas à equipe médica para o tratamento dos pacientes.

Os pacientes que sofrem de problemas de saúde mental podem, dada a natureza de seus problemas, perder consultas, receitas e medicações — de modo deliberado ou acidental —, esquecer instruções e fazer pedidos sem sentido à equipe médica. Eles podem aparecer nas clínicas inesperadamente. Em uma minoria dos casos, podem ser perigosos para si próprios ou para outras pessoas. Podem mudar regularmente de endereço ou podem ser sem-teto a pouco ou muito tempo. Quando os pacientes são perigosos, podem precisar ser internados em um hospital seguro para tratamento e observação.

Os usuários do sistema incluem o pessoal das clínicas, como médicos, profissionais de enfermagem e assistentes de saúde (profissionais de enfermagem que visitam as pessoas em casa para acompanhar o seu tratamento). Os usuários que não são da área médica incluem recepcionistas que marcam consultas, profissionais responsáveis pela manutenção do sistema de cadastros e pessoal administrativo que gera os relatórios.

O sistema é utilizado para registrar informações sobre os pacientes (nome, endereço, idade, familiares etc.), consultas (data, médico consultado, impressões subjetivas do paciente etc.), condições e tratamentos. Os relatórios são gerados em intervalos regulares para a equipe médica e os gestores de saúde autorizados. Normalmente, os relatórios para a equipe médica se concentram nas informações de cada paciente, enquanto que os relatórios de gestão são anônimos e se preocupam com condições, custos de tratamento etc.

As características³ principais desse sistema são:

1. *Gestão do cuidado individual.* Os médicos podem criar registros para os pacientes, editar as informações no sistema, visualizar o histórico do paciente etc. O sistema dá suporte ao resumo de dados, de modo que os médicos que ainda não conheçam o paciente possam aprender rapidamente sobre seus principais problemas e os tratamentos prescritos.
2. *Monitoramento do paciente.* O sistema monitora regularmente os registros dos pacientes que estão sendo tratados e emite alertas se forem detectados possíveis problemas. Portanto, se um paciente não foi atendido por um médico durante certo intervalo de tempo, pode ser emitido um alerta. Um dos elementos mais importantes do sistema de monitoramento é acompanhar os pacientes que foram internados e assegurar que sejam realizadas as averiguações exigidas legalmente, no momento certo.
3. *Relatórios administrativos.* O sistema gera relatórios mensais de gestão mostrando a quantidade de pacientes tratados em cada clínica, a quantidade de pacientes que entrou e saiu do sistema de atendimento, a quantidade de pacientes internados, os medicamentos prescritos e seus custos etc.

Dois tipos de leis diferentes afetam o sistema: leis de proteção de dados, que governam a confidencialidade das informações pessoais; e leis de saúde mental, que governam a internação compulsória dos pacientes considerados uma ameaça para si próprios ou para outros. A saúde mental diferencia-se das demais especialidades por ser a única que permite a um médico recomendar a internação dos pacientes contra a sua vontade. Isso está sujeito a garantias legislativas rigorosas. Um objetivo do sistema Mentcare é garantir que a equipe aja sempre em concordância com a lei e que suas decisões sejam registradas para análise judicial, caso seja necessário.

Assim como em todos os sistemas clínicos, a privacidade é um requisito crítico do sistema. É essencial que as informações dos pacientes sejam confidenciais e que jamais sejam reveladas a qualquer outra pessoa além da equipe médica autorizada e dos próprios pacientes. O Mentcare também é um sistema crítico em segurança. Alguns transtornos mentais fazem com que os pacientes se tornem suicidas ou uma ameaça para as outras pessoas. Sempre que possível, o sistema adverte a equipe médica sobre os pacientes potencialmente suicidas ou perigosos.

O projeto geral do sistema precisa levar em conta os requisitos de privacidade e segurança (*safety*). Ele deve estar disponível quando necessário, ou a segurança pode

³ N. da R.T.: algumas pessoas preferem usar a palavra *feature*, sem tradução.

ficar comprometida e pode ser impossível prescrever a medicação correta para os pacientes. Aqui temos um conflito em potencial — a privacidade é mais fácil de manter quando há apenas uma única cópia dos dados do sistema. No entanto, para assegurar a disponibilidade no caso de falhas do servidor ou quando não houver conexão de rede, devem ser mantidas múltiplas cópias dos dados. Discuto os conflitos de escolha entre esses requisitos nos capítulos posteriores.

1.3.3 Estação meteorológica na natureza

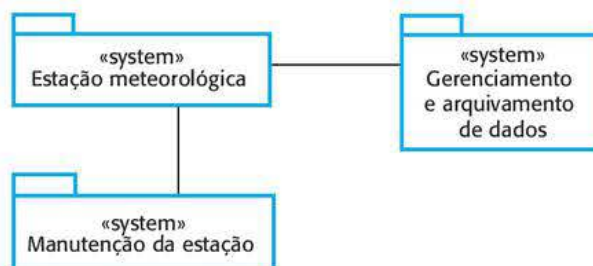
Para ajudar a monitorar as mudanças climáticas e tornar mais precisas as previsões meteorológicas nas áreas remotas, o governo de um país com grandes áreas de natureza selvagem decide instalar centenas de estações meteorológicas em áreas remotas. Essas estações coletam dados de um conjunto de instrumentos que medem a temperatura e a pressão, a insolação, as precipitações e a velocidade e direção dos ventos.

As estações meteorológicas na natureza fazem parte de um sistema maior (Figura 1.7), que é o sistema de informações climáticas que coleta dados das estações e os disponibiliza para outros sistemas para que sejam processados. Os sistemas apresentados na Figura 1.7 são:

1. *Sistema da estação meteorológica.* É responsável por coletar dados climáticos, realizar algum processamento inicial e transmitir os dados para o sistema de gerenciamento.
2. *Sistema de gerenciamento e arquivamento de dados.* Coleta dados de todas as estações meteorológicas, processa e analisa os dados e os arquiva de uma forma que possam ser recuperados por outros sistemas, como os sistemas de previsão meteorológica.
3. *Sistema de manutenção da estação.* Capaz de se comunicar via satélite com todas as estações meteorológicas na natureza para monitorar a condição desses sistemas e fornecer relatórios dos problemas. Ele também pode atualizar o software embarcado nesses sistemas. No caso de problemas de sistema, ele pode ser utilizado para controlar remotamente a estação meteorológica.

Na Figura 1.7, utilizei o símbolo de pacote da UML para indicar que cada sistema é um conjunto de componentes, e os diferentes sistemas são identificados usando o estereótipo «system» da UML. As associações entre os pacotes indicam que há uma troca de informações, mas, nesse estágio, não há necessidade de definir esses pacotes com mais detalhes.

FIGURA 1.7 O ambiente da estação meteorológica.



As estações meteorológicas incluem instrumentos que medem os parâmetros climáticos, como velocidade e direção do vento, temperaturas no solo e no ar, pressão barométrica e precipitações ao longo de um período de 24 horas. Cada um desses instrumentos é controlado por um software que recebe periodicamente as leituras dos parâmetros e gerencia os dados coletados.

O sistema da estação meteorológica funciona coletando observações climáticas em intervalos frequentes; as temperaturas, por exemplo, são medidas a cada minuto. No entanto, como a largura de banda da conexão por satélite é insuficiente, a estação meteorológica realiza parte do processamento e agregação dos dados localmente. Depois, ela transmite esses dados agregados quando solicitado pelo sistema de coleta de dados. Se for impossível estabelecer uma conexão, então a estação meteorológica mantém os dados localmente até a comunicação ser restabelecida.

Cada estação meteorológica é alimentada por baterias e deve ser inteiramente autocontida; não há alimentação externa ou cabos de rede. Todas as comunicações se dão através de um *link* via satélite relativamente lento, e a estação meteorológica deve incluir algum mecanismo (solar ou eólico) para carregar as baterias. Como são instaladas em áreas selvagens, as estações ficam expostas a condições ambientais severas e podem ser danificadas por animais. O software da estação, portanto, não está preocupado apenas com a coleta de dados. Ele também deve:

1. Monitorar os instrumentos, a alimentação e o hardware de comunicação e relatar defeitos para o sistema de gerenciamento.
2. Gerenciar a alimentação do sistema, garantindo que as baterias sejam carregadas sempre que as condições ambientais permitirem, mas que os geradores sejam desligados em condições climáticas potencialmente danosas, como ventos fortes.
3. Permitir a reconfiguração dinâmica quando partes do software forem substituídas por novas versões e quando os instrumentos de *backup* forem conectados em caso de falha de sistema.

Uma vez que as estações meteorológicas precisam ser autocontidas e independentes, o software instalado é complexo, embora a funcionalidade da coleta de dados seja bem simples.

1.3.4 Ambiente digital de aprendizagem para escolas

Muitos professores argumentam que usar sistemas de software interativos na educação pode motivar quem está aprendendo e gerar níveis de conhecimento e compreensão mais acentuados. No entanto, não há consenso quanto à 'melhor' estratégia para a aprendizagem com suporte do computador e os professores usam, na prática, uma gama de diferentes ferramentas web interativas para apoiar a aprendizagem. As ferramentas utilizadas dependem das idades dos estudantes, de sua bagagem cultural, de sua experiência com computadores, do equipamento disponível e das preferências dos professores envolvidos.

Um ambiente digital de aprendizagem é um *framework* ao qual um conjunto de ferramentas de propósito geral e ferramentas especialmente projetadas para a aprendizagem pode ser incorporado, além de um conjunto de aplicações voltadas para as necessidades dos estudantes que usam o sistema. O *framework* fornece serviços gerais como serviço de autenticação, serviços de comunicação síncrona e assíncrona e serviço de armazenamento.

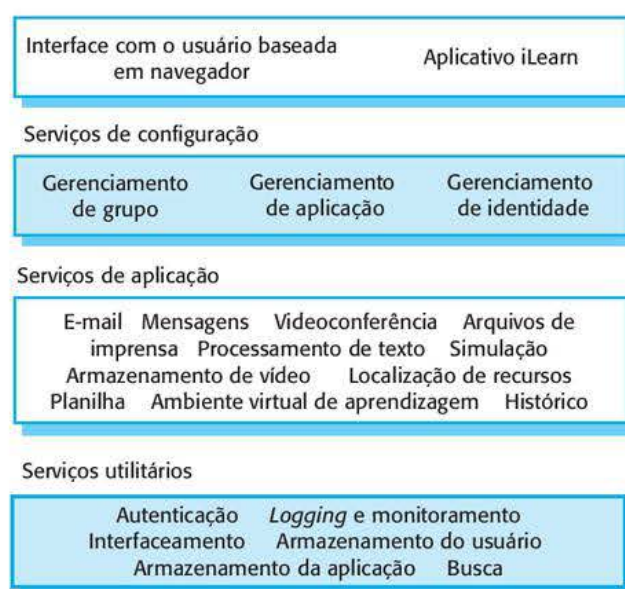
As ferramentas incluídas em cada versão do ambiente são escolhidas pelos professores e estudantes de acordo com suas necessidades. Elas podem ser aplicações genéricas, como planilhas, aplicações de gestão do aprendizado — um Ambiente Virtual de Aprendizagem (AVA) para gerenciar o envio e a avaliação das tarefas de casa —, jogos e simulações. Também podem incluir conteúdo específico — como as informações a respeito da guerra civil norte-americana — e aplicações para visualizar e anotar o conteúdo.

A Figura 1.8 é um modelo de arquitetura de alto nível de um ambiente digital de aprendizagem (iLearn) que foi concebido para utilização em escolas, por alunos dos 3 aos 18 anos de idade. A abordagem adotada é a de um sistema distribuído em que todos os componentes do ambiente são serviços que podem ser acessados de qualquer lugar através da internet. Não é necessário que todas as ferramentas de aprendizagem estejam reunidas em um único lugar.

O sistema é orientado a serviços, com todos os componentes considerados serviços substituíveis. Existem três tipos de serviço no sistema:

1. *Serviços utilitários*, que fornecem funcionalidade, independentemente da aplicação básica, e que podem ser utilizados por outros serviços no sistema. Os serviços utilitários normalmente são desenvolvidos ou adaptados especificamente para este sistema.
2. *Serviços de aplicação*, que fornecem as aplicações específicas como e-mail, videoconferência, compartilhamento de fotos etc., e acesso ao conteúdo educacional específico, como filmes científicos ou recursos históricos. Os serviços de aplicação são externos e adquiridos especificamente para o sistema ou disponibilizados gratuitamente pela internet.
3. *Serviços de configuração*, que são utilizados para adaptar o ambiente a um conjunto específico de serviços de aplicação e para definir como os serviços são compartilhados entre alunos, professores e seus pais.

FIGURA 1.8 A arquitetura de um ambiente digital de aprendizagem (iLearn).



O ambiente foi projetado para que os serviços possam ser substituídos à medida que novos serviços forem disponibilizados e para proporcionar diferentes versões do sistema que sejam adequadas à idade dos usuários. Isso significa que o sistema tem de suportar dois níveis de integração de serviço:

1. *Serviços integrados.* São aqueles que oferecem uma API (interface de programação da aplicação, do inglês *application programming interface*) e que podem ser acessados por outros serviços através dessa API. Portanto, a comunicação direta entre serviços é possível. Um serviço de autenticação é um exemplo de serviço integrado, pois em vez de usar mecanismos de autenticação próprios, ele pode ser chamado por outros serviços para autenticar usuários. Se os usuários já estiverem autenticados, então o serviço de autenticação pode passar a informação de autenticação diretamente para outro serviço, por meio de uma API, sem que os usuários precisem ser autenticados novamente.
2. *Serviços independentes.* São aqueles acessados por meio de uma interface do navegador e que operam de maneira independente dos demais serviços. As informações só podem ser compartilhadas com outros serviços por ações explícitas do usuário, como copiar e colar; a reautenticação pode ser necessária para cada serviço independente.

Se um serviço independente passar a ser amplamente utilizado, o time de desenvolvimento pode torná-lo um serviço integrado de modo que esteja sujeito a suporte.

PONTOS-CHAVE

- ▶ A engenharia de software é uma disciplina de engenharia que se preocupa com todos os aspectos da produção de software.
- ▶ O software não é apenas um programa ou conjunto de programas; também faz parte do software toda a documentação eletrônica necessária para os usuários do sistema, a equipe de garantia da qualidade e os desenvolvedores. Os atributos essenciais do produto de software são a manutenibilidade, a dependabilidade e a segurança da informação (*security*), a eficiência e a aceitabilidade.
- ▶ O processo de software inclui todas as atividades envolvidas no desenvolvimento do software. As atividades de alto nível, como especificação, desenvolvimento, validação e evolução fazem parte de todos os processos de software.
- ▶ Existem muitos tipos diferentes de sistema e cada um deles requer ferramentas e técnicas de engenharia de software adequadas para o seu desenvolvimento. Poucas técnicas (ou nenhuma) de projeto e implementação são aplicáveis a todos os tipos de sistema.
- ▶ As ideias fundamentais de engenharia de software são aplicáveis a todos os tipos de sistema de software. Esses fundamentos incluem os processos de software gerenciados, a dependabilidade e segurança da informação (*security*), engenharia de requisitos e reúso de software.
- ▶ Os engenheiros de software têm responsabilidade com a profissão de engenharia e com a sociedade. Além de se preocupar com questões técnicas, devem estar a par das questões éticas que afetam o seu trabalho.
- ▶ As sociedades profissionais publicam códigos de conduta que definem padrões éticos e profissionais. Esses códigos estabelecem os padrões de comportamento esperados de seus membros.

LEITURAS ADICIONAIS

"Software engineering Code of Ethics is approved." Um artigo que discute a fundamentação do desenvolvimento do código de ética da ACM/IEEE e que inclui a forma abreviada e a completa do código. GOTTERBARN, D.; MILLER, K.; ROGERSON, S. *Comm. ACM*. Out., 1999. doi:10.1109/MC.1999.796142.

"A view of 20th and 21st century software engineering." Uma visão retrospectiva e prospectiva da engenharia de software por um dos mais destacados engenheiros de software. Barry Boehm identifica princípios atemporais da engenharia de software, mas também sugere que algumas práticas utilizadas frequentemente estão obsoletas. BOEHM, B. *Proceedings of the 28th International Conference on Software Engineering*. Shanghai, 2006. doi:10.1145/1134285.1134288.

"Software engineering ethics." Edição especial do *IEEE Computer*, com vários artigos sobre o tema. *IEEE Computer*. v. 42, n. 6, jun. 2009.

Ethics for the Information Age. Um livro abrangente que cobre todos os aspectos da ética na tecnologia da informação (TI), e não apenas a ética para os engenheiros de software. Acredito que esta seja a abordagem correta, já que entender a ética da engenharia de software dentro de um arcabouço ético mais amplo é necessário. QUINN, M. J. Addison-Wesley, 2013.

The essence of software engineering: applying the SEMAT kernel. Este livro discute a ideia de um *framework* universal que pode estar por trás de todos os métodos de engenharia de software; ele pode ser adaptado e utilizado para todos os tipos de sistemas e organizações. Eu sou particularmente cético quanto à abordagem universal ser ou não realista na prática, mas o livro tem algumas ideias interessantes que valem a exploração. JACOBSON, I.; NG, P-W.; MCMAHON, P. E.; SPENCE, I.; LIDMAN, S. Addison-Wesley, 2013.

SITE⁴

Apresentações em PowerPoint para este capítulo disponíveis em: <<http://software-engineering-book.com/slides/chap1/>>.

Links para vídeos de apoio disponíveis em: <<http://software-engineering-book.com/videos/software-engineering/>>.

Links para descrições dos estudos de caso disponíveis em: <<http://software-engineering-book.com/case-studies/>>.

⁴ Todo o conteúdo disponibilizado na seção *Site* de todos os capítulos está em inglês.

EXERCÍCIOS

- 1.1. Explique por que o software profissional desenvolvido para um cliente não consiste apenas nos programas que foram desenvolvidos e fornecidos.
- 1.2. Qual é a diferença mais importante entre desenvolvimento de software genérico e desenvolvimento de software personalizado? O que isso poderia significar na prática para os usuários dos produtos de software genéricos?
- 1.3. Quais são os quatro atributos importantes que todo o software profissional possui? Sugira outros quatro atributos que às vezes podem ser relevantes.
- 1.4. Além dos desafios de heterogeneidade, de mudanças sociais e de negócio e de confiança e segurança da informação (*security*), sugira outros problemas e desafios que a engenharia de software tende a enfrentar no século XXI. (Dica: pense no meio ambiente.)
- 1.5. Com base no seu próprio conhecimento de alguns dos tipos de aplicação discutidos na Seção 1.1.2, explique, com exemplos, por que os diferentes tipos de aplicação necessitam de técnicas especializadas de engenharia de software para dar suporte ao seu projeto e desenvolvimento.
- 1.6. Explique por que os princípios fundamentais da engenharia de software, como processo, dependabilidade, gerenciamento de requisitos e reuso, são relevantes para todos os tipos de sistema de software.
- 1.7. Explique como o uso universal da web mudou os sistemas de software e a sua engenharia.
- 1.8. Discuta se os engenheiros profissionais devem ser licenciados da mesma maneira que os médicos e os advogados.
- 1.9. Para cada uma das cláusulas no código de ética da ACM/IEEE exibido na Figura 1.3, proponha um exemplo adequado que ilustre essa cláusula.
- 1.10. Para ajudar a combater o terrorismo, muitos países estão planejando ou desenvolveram sistemas de computador que rastreiam um grande número de seus cidadãos e suas ações, o que, claramente, tem implicações na privacidade. Discuta a ética de trabalhar no desenvolvimento desse tipo de sistema.

REFERÊNCIAS

- BOTT, F. *Professional issues in information technology*. Swindon, UK: British Computer Society, 2005.
- DUQUENOY, P. *Ethical, legal and professional issues in computing*. London: Thomson Learning, 2007.
- FREEMAN, A. *The definitive guide to HTML5*. New York: Apress, 2011.
- GOTTERBARN, D.; MILLER, K.; ROGERSON, S. "Software engineering Code of Ethics is approved." *Comm. ACM*, v. 42, n. 10, 1999. p. 102-107. doi:10.1109/MC.1999.796142.
- HOLDENER, A. T. *Ajax: the definitive guide*. Sebastopol, CA: O'Reilly and Associates, 2008.
- JACOBSON, I.; NG, P-W.; MCMAHON, P. E.; SPENCE, I.; LIDMAN, S. *The essence of software engineering*. Boston: Addison-Wesley, 2013.
- JOHNSON, D. G. *Computer ethics*. Englewood Cliffs, NJ: Prentice-Hall, 2001.
- LAUDON, K. "Ethical concepts and information technology." *Comm. ACM*, v. 38, n. 12, 1995. p. 33-39. doi:10.1145/219663.219677.
- NAUR, P.; RANDELL, B. "Software engineering: report on a conference sponsored by the NATO Science Committee." Brussels, 1969. Disponível em: <<http://homepages.cs.ncl.ac.uk/brian.randell/NATO/NATOReports/>>. Acesso em: 25 mar. 2018.
- TAVANI, H. T. *Ethics and technology: controversies, questions, and strategies for ethical computing*, 4th ed. New York: John Wiley & Sons, 2013.
- VOGEL, L. *Eclipse 4 application development: the complete guide to Eclipse 4 RCP development*. Sebastopol, CA: O'Reilly & Associates, 2012.

